

Efficient and Practical Solutions for Correlated Storage Location Assignment Problem in Automated Warehouse Environment

Abstract

Traditional approaches to the Correlated Storage Location Assignment Problem (CSLAP) focus heavily on minimizing physical travel distance. However, in automated, conveyor-driven zone-picking architectures, operational bottlenecks are dictated by system congestion, mechanical dwell times, and workload imbalances, rather than pure travel distance. This paper proposes a paradigm shift by modeling CSLAP with the explicit objective of minimizing the total number of station visits per order. We introduce a rigorous mathematical framework featuring a Mixed-Integer Linear Program (MILP) bound by strict workload synchronization constraints, a scalable clustering heuristic, and a Column Generation (CG) approach. By evaluating these methodologies against classic meta-heuristic benchmarks across both synthetic datasets and a large-scale industrial instance (21,877 SKUs), we demonstrate that explicitly penalizing network dispersion under strict capacity constraints significantly reduces system queuing times and increases throughput.

Keywords: Warehouse optimization, Integer programming, Heuristics, Column generation, Logistics

1. Introduction

Efficient warehouse operations significantly influence the performance of supply chains. Order picking alone typically accounts for 50–65% of warehouse operating costs [?], making optimized storage assignment crucial. Central to this optimization challenge is the Correlated Storage Location Assignment Problem (CSLAP), which focuses on strategically placing products to streamline order preparation and fulfillment.

Traditional CSLAP models primarily aim to *minimize travel distance* during order preparation [?]. However, in automated, conveyor-driven picking systems, and zone-picking architectures, *reducing the number of station stops* is the absolute operational bottleneck. Each additional conveyor stop introduces mechanical setup time, conveyor dwell time, and queuing delays. From an analytical perspective, a box’s total preparation time consists of a fixed conveyor transition time, an average queue transition time at each station, and the operator processing time depending on the products handled. Reducing the number of station visits per order inherently minimizes the times a box is inducted and merged back into the main conveyor, significantly reducing its average queuing and mechanical dwell time. By explicitly targeting station-stop reduction, our approach addresses this critical dimension, minimizing congestion and accelerating fulfillment processes.

In this paper, we advance the state of the art by proposing a rigorous mathematical framework specifically tailored to automated environment dynamics. We propose three methods to tackle this modern CSLAP: (i) An exact approach through mixed-integer linear program (MILP) that minimizes station visits while enforcing strict synchronization constraints and balanced processing times across automated stations, (ii) a fast heuristic clustering approach that groups frequently co-ordered products, and (iii) a Column Generation (CG) framework designed to scale the exact approach by decomposing CSLAP into a master problem selecting optimal patterns and multiple pricing subproblems enforcing station capacity limits. Furthermore, recognizing that established facilities cannot afford large, disruptive re-slotting waves, we extend the MILP with a *limited-reassignment* mech-

anism that limits the permitted relocation while maximizing continuous-improvement gains [?].

The primary contribution of this research is twofold: the **rigorous mathematical modeling of station-visit minimization under strict workload balancing constraints**, and the **design of a scalable heuristic/CG framework** tailored for complex automated picking environments. We validate the industrial applicability and computational limits of this theoretical framework using both synthetic, diverse demand scenarios and a massive real-world dataset extracted from Company A.

This work largely extends preliminary results presented earlier in a peer-reviewed conference paper, details are withheld for double anonymized review and will be reinstated after acceptance.

2. Literature Review

The transition from traditional, manual picker-to-parts warehouses to highly automated, conveyor-driven, and synchronized zone-picking systems necessitates a fundamental paradigm shift in operations research modeling. Historically, the Correlated Storage Location Assignment Problem (CSLAP) has been approached almost exclusively through the lens of distance minimization [?]. In manual systems, where human operators traverse vast arrays of shelving, the primary operational cost driver is indeed the physical travel distance. However, applying this identical objective function to automated conveyor-based systems, pick-and-pass architectures, or Robotic Mobile Fulfillment Systems (RMFS) represents a categorical fallacy in system dynamics. In these automated environments, the physical movement of goods across long distances is largely mechanized and highly efficient. Consequently, the true operational bottlenecks and systemic cost drivers shift away from human travel distance and manifest as system-level metrics: station visits (stops), order splits, dwell time, and the synchronization of workload across discrete, capacity-constrained picking zones [?].

This section provides a rigorous overview extending from classical CSLAP to the operational realities of automated zone-picking architectures, exam-

ining state-of-the-art mathematical formulations that specifically target the minimization of station stops and the strict balancing of station capacity.

2.1. The Architectural Shift: From Distance Minimization to Flow Synchronization

The Cost of Station Stops and Dwell Time: In a pick-and-pass conveyor system, every time a tote must divert from the main conveyor to a picking station, a mechanical and operational setup time is incurred. The tote must be decelerated, inducted into the station via a transfer matrix or diverter, processed by the picker, and then merged back into the main traffic line. Minimizing the number of zones an order must visit minimizes these setup times, reduces dwell time, and drastically increases conveyor throughput [?].

Queueing Theory, Starvation, and Blocking: A pick-and-pass conveyor is effectively a network of interconnected queues. Each zone functions as a server with a specific service rate, and the conveyor delivers totes at a specific arrival rate. If a traditional, distance-centric CSLAP optimization is applied to a conveyor system, it tends to group all highly correlated items tightly together, resulting in placing the vast majority of the workload into a single zone. As the arrival rate approaches the picker's service rate, the expected wait time grows exponentially. Because the physical spur conveyor has a finite buffer capacity, the queue will quickly overflow, causing "blocking". Thus, CSLAP formulations must carefully target workload balancing, as it is the prerequisite for avoiding starvation and blocking [?].

2.2. Objectivity Reframing in Automated Systems: Visits and Splits

Modern literature on automated warehousing explicitly supports the transition away from distance-metrics towards minimizing discrete touches or stops. [?] provides a seminal exploration of minimizing order splits across multiple warehouses using clustering algorithms, which directly translates to clustering items into discrete picking zones along a conveyor to maintain order integrity.

In the realm of Robotic Mobile Fulfillment Systems (RMFS) [?], the realities are highly homologous to a conveyor pick-and-pass system. [?] presented a rigorous Mixed-Integer Programming (MIP) model whose primary objective is the minimization of "pod-station visits". They explicitly modeled split orders and proved mathematically that minimizing visits while dispersing workload across stations increases total system throughput. Similarly, [?] proposed "correlated dispersed storage assignment," distributing highly correlated items intelligently across available automated assets to prevent queueing bottlenecks at any single workstation.

2.3. Workload Balancing: The Hard Constraint in Conveyor Operations

[?] directly addressed the integration of workload balancing in pick-and-pass warehousing, offering a comprehensive MILP model that optimizes storage location specifically with the hard constraint of balancing workload across all picking zones. The model proved that mathematically balancing the workload significantly reduced the total picking time. [?] focused on order batching with Group Genetic Algorithms (GGA), attempting to balance the workload of each zone while minimizing the total batches releases into the system. [?] utilized ANOVA to prove that storage assignment cannot be optimized in isolation; it must be co-optimized with zoning logic to suppress dwell time. Ultimately, [?] concluded that ensuring the right tote arrives at a station without queueing blockages is the ultimate determinant of facility throughput.

2.4. Advanced Exact Methods: Column Generation and MILP

Because of the NP-hard nature of integrating storage assignment with workload synchronization, exact methods have gained prominence. [?] utilized Column Generation (CG) formulated as Set Partitioning to minimize the makespan, enforcing workload pacing as resource limits within the Pricing Problem (ESPPRC). Furthermore, [?] devised advanced column- and row-generation algorithms to explicitly handle the hard synchronization constraints. By utilizing exact generation frameworks, modern researchers

effectively handle station capacity and synchronization as rigid mathematical bounds, fundamentally validating our approach of utilizing MILP and CG to handle the CSLAP with station-visit minimization.

3. Formal Problem Definition

Our case study examines the Correlated Storage Location Assignment Problem (CSLAP) on a real-world instance using historical data provided by Company A, which operates an automated warehouse. The initial placement of products (P) within this warehouse was arranged in the best possible way by operators, drawing upon their extensive practical knowledge and daily operational expertise.



Figure 1: Picking Section Illustration

The system is organized around a main conveyor along which orders advance in sequence. Branch lines off the main conveyor correspond to individual picking stations (S). At each station ($s \in S$), an order ($o \in O$) temporarily leaves the main conveyor so that human pickers can retrieve the required items from nearby shelving and place them into the order bin. After the station's items are collected, the bin returns to the main conveyor and proceeds its journey. This process repeats at subsequent stations until

all items in the order (P_o) are collected. Finally, the completed bin exits through the main conveyor output, ready for dispatch.

In this setting, limiting the number of station stops per order is essential, since each stop significantly contributes to the total order preparation time due to waiting and queuing at the stations. Our aim is to demonstrate effectiveness under operational conditions in an actual warehouse. We make use of some historical data from Company A (with $|P| = 21\,877$, $|O| = 284\,871$, and $|S| = 26$). Because our dataset records, for every order, the *order ID*, the corresponding *product IDs*, the current *station ID*, and the overall *number of pick locations available at each station*, we model CSLAP at the station level rather than at the level of individual storage slots. We make the practical assumption that each station’s internal shelving is *modular*: dividers can be rearranged without altering the station’s *total* storage capacity. This intra-station flexibility allows us to concentrate on the key decision, *which products* ($p \in P$) *should share the same station*, without being restricted by fixed slot geometries. Throughout our analysis, the global capacity constraint (total number of locations per station) is strictly respected, even though the internal configuration of those locations may be rebalanced.

3.1. Evaluation Metrics

We evaluate against the company’s original assignment using:

(1) Number of orders per number of visited stations: Illustrates for each possible number of visited stations the number of orders which has visited exactly this number of stations.

(2) Daily Station Utilization (Workload Balance): Assesses workload distribution by quantifying the temporal utilization required at each station per day. We compute the assigned daily utilization factor T'_s (expressed as a fraction of the working day) based on the division of the average daily pick-lines L_p by the station’s daily processing speed V_s . We illustrate the *relative change* in temporal workload compared to the company’s original assignment, computed as:

$$\text{Relative Change (\%)} = \frac{T'_s - T_s}{T_s} \times 100$$

where T_s denotes the original daily utilization baseline allocated to station s under Company A’s assignment. This percentage highlights how significantly each station’s processing load is impacted relative to its historical baseline, ensuring operational equilibrium is maintained.

3.2. Workload Balancing: The Hard Constraint in Heterogeneous Environments

A pick-and-pass conveyor functions as a network of interconnected queues. If a traditional CSLAP optimization groups highly correlated items too tightly into a single zone, the arrival rate of bins approaches the picker’s service rate. This causes expected wait times to grow exponentially, eventually leading to starvation and network blocking [? ?].

In practice, preventing this requires acknowledging that real-world warehouses are highly heterogeneous. Company A’s stations differ markedly in physical size, slot depth, and inherent picker productivity. Because of these structural differences, Company A has deliberately calibrated its legacy layout to keep labor demand, travel effort, and throughput in equilibrium across the facility. Preserving this equilibrium is essential for smooth operations. Therefore, in our formulation, a “balanced workload” acts as a hard constraint: it refers to enforcing a daily utilization distribution that stays strictly within the company’s pre-established baseline capacity (T_s). This ensures that while we reduce station visits, no single station is pushed into over-utilization after re-slotting.

3.3. Analytical Queueing Justification

To formally justify that minimizing station stops improves throughput, we abstract the conveyor line as a network of M/M/1 queues. A box traversing the system incurs a total operational time W_{total} , composed of a fixed conveyor traversal time W_{conv} , and the times spent at each visited station s :

$$W_{total} = W_{conv} + \sum_{s \in visited} (W_{queue,s} + W_{prep,s}) \quad (1)$$

Every visited station strictly adds a waiting penalty $W_{queue,s}$ regardless of the product volume retrieved. However, aggressively minimizing the cardinality of the visited set without constraints will route all traffic to a few centralized stations. This would drive a station’s required processing time beyond its safe operational limit, causing $W_{queue,s} \rightarrow \infty$. Therefore, minimizing station visits must be mathematically coupled with the strict workload-balancing constraint, bounding the assigned processing load to the station’s calibrated daily capacity (T_s) to guarantee a true reduction in W_{total} .

Symbol	Definition
P, O, S	Sets of products, orders, and stations
P_o	Set of products requested in order $o \in O$
ζ_s	Number of available storage locations at station $s \in S$
L_p	Average daily number of pick-lines for product $p \in P$ (lines/day)
V_s	Processing speed at station $s \in S$ (lines/day)
T_s	Maximum daily time capacity allocated to station s (fraction of a day)
K_s	Set of feasible patterns (product assignments) for station $s \in S$

Summary of key notations

4. Methodology

4.1. First Approach: Mixed-Integer Linear Programming (MILP)

To structurally and formally address the Correlated Storage Location Assignment Problem (CSLAP) in an automated context, we first developed a Mixed-Integer Linear Programming (MILP) model. This exact formulation simultaneously minimises station visits and enforces strict, balanced workloads. Building on the demand-correlation ideas of [?], who focused on minimizing travel distance, we fundamentally redirected the objective toward minimizing the *number of station stops*. Orders thus flow through fewer stations, systematically cutting mechanical queueing and handling time.

The formulation employs binary decision variables (DVs) to encode product–station assignments and auxiliary variables to record, for each order, the

stations it visits. The formulation ensures capacity constraints, workload balancing, and station visit counting, which is internally consistent with the corresponding product-to-station assignments.

Decision variables:

- x_{ps} : Binary variable indicating whether product $p \in P$ is assigned to station $s \in S$.
- z_{os} : Auxiliary variable indicating whether order $o \in O$ needs to visit station $s \in S$ (completely dependent on x_{ps}).

Objective function:

$$\min \sum_{o \in O} \sum_{s \in S} z_{os} \quad (2)$$

The objective is to minimize the total number of station visits required across all orders.

Constraints:

$$\sum_{s \in S} x_{ps} = 1 \quad \forall p \in P \quad (3)$$

$$\sum_{p \in P} x_{ps} \leq \zeta_s \quad \forall s \in S \quad (4)$$

$$x_{ps} \leq z_{os} \quad \forall p \in P_o, \forall o \in O, \forall s \in S \quad (5)$$

$$\sum_{p \in P_o} x_{ps} \cdot \frac{L_p}{V_s} \leq T_s \quad \forall s \in S \quad (6)$$

$$x_{ps} \in \{0, 1\}, \quad z_{os} \in [0, 1], \quad \forall p \in P, o \in O, s \in S \quad (7)$$

Here, Constraint (??) ensures each product is assigned to exactly one station, while Constraint (??) respects the physical capacity limit of each station. Constraint (??) ensures z_{os} is mapped exactly to the visits (acting as a logical OR without needing a native Big-M formulation since the objective natively minimizes $\sum z_{os}$). Finally, Constraint (??) enforces a balanced workload on each station.

4.2. Second Approach: Fast Heuristic Formulation

Because the exact MILP formulation becomes computationally intractable for large-scale, industrial-sized warehouse instances (where the combination of millions of orders and tens of thousands of products explodes the decision variables), we formulated a fast heuristic approach. The heuristic reduces station visits quickly through practical product clustering, though it may leave some minor workloads uneven across stations compared to the strict MILP constraints.

The method draws on graph-connectivity ideas [?] to build subsets of strongly co-occurring products that should be co-located at the same station. Using greedy selection with simple decision rules, we rank and include product pairs with the highest co-occurrence frequencies.

4.2.1. Heuristic Approach: Greedy Product Clustering

We partition the product set into “communities” of highly correlated items and then assign these communities to stations. Here, we will present some new notations to define different thresholds we have used for our heuristic process:

- **MIN_FREQ_PREPROC**: Defines the minimum frequency that a product appears, where **frequency** refers to how many distinct orders contain the product, independently of the ordered quantity.
- **MIN_FREQ**: Minimum co-occurrence for pairs of products to be considered relevant.
- **RATIO_TO_KEEP**: Minimum fraction of co-occurrence relative to a product’s total frequency.
- **MNOPPC**: Maximum number of products allowed in a single community.

The core algorithm of our heuristic is:

Algorithm 1: Greedy Product Clustering and Station Assignment (CSLAP)

Step 1: Preprocessing and Pair Generation.

- (a) Define $P^* = \{p \in P : \text{freq}(p) \geq \text{MIN_FREQ_PREPROC}\}$.
 - (b) For each multi-item order $o \in O$, generate all unordered pairs (p_i, p_j) with $p_i, p_j \in P^*$ and $i < j$.
 - (c) Compute the co-occurrence count : $\text{count}_{ij} = |\{o \in O : \{p_i, p_j\} \subset o | i < j\}|$, and keep only pairs with $\text{count}_{ij} \geq \text{MIN_FREQ}$.
 - (d) For each kept pair, compute $\eta_{ij} = \text{count}_{ij} / \text{count}_i$ and retain those with $\eta_{ij} \geq \text{RATIO_TO_KEEP}$.
 - (e) Sort remaining pairs in descending count_{ij} (ties arbitrary).
-

Step 2: Community Formation.

- (a) Initialize the set of communities $\mathcal{C} \leftarrow \emptyset$; mark all products as “unassigned”.
- (b) While there are candidate pairs left:
 - (b)1. Take the top pair (p_a, p_b) and start a community $c = \{p_a, p_b\}$.
 - (b)2. *Iterative expansion:* while $|c| < \text{MNOPPC}$, select the single product $p_x \notin c$ that has the highest aggregate co-occurrence with the products already in c ; add p_x to c .
 - (b)3. After adding p_x , remove from the candidate list every pair that contains p_x (so no product can join multiple communities).
 - (b)4. Stop expansion if no suitable pair remains or the size limit MNOPPC is reached.
 - (b)5. *Repeat ...*

Step 3: Post-Processing.

- (a) *Low-frequency products:* for any $p \in \overline{P^*}$, if some community c has room ($|c| < \text{MNOPPC}$), place p into the c with which it has the highest co-occurrence.
- (b) *Residuals:* if any products remain unassigned, group them into a new community (respecting MNOPPC as needed).

Step 4: Station Assignment.

- (a) Order communities by their cumulative product frequency (most impactful first).
- (b) For each community c , compute historical station preferences from past assignments of products in c ; sort stations by this preference.
- (c) *Majority-preference placement:* assign c to the first preferred station s with enough available capacity ζ_s to host all products in c .
- (d) If no preferred station can host c entirely, try any other station with sufficient capacity.
- (e) If no single station can host c , split c across stations: first along preferred stations (in order), then by remaining capacity, maximizing how many correlated products stay together.

This heuristic approach represents a balance between computational feasibility and the attainment of a near-optimal solution. By addressing the complexities inherent in CSLAP, it offers a scalable and flexible framework capable of adapting to evolving order patterns and warehouse configurations, setting a new standard for optimizing warehouse operations.

4.3. Column Generation Approach

While the heuristic provides fast and scalable solutions, exact mathematical bounds are ultimately required for rigorous structural assessments. To circumvent the direct computational limitations of the exact MILP formulation on scale, we propose a Column Generation (CG) framework to reliably deliver high-quality bounded solutions more quickly. CG efficiently solves combinatorial optimization problems involving vast solution spaces [?], making it computationally superior to monolithic ILPs for set-covering and set-partitioning problems in logistics, scheduling, and warehouse management [?].

Set Cover and its dual, Hitting Set, are classic NP-complete combinatorial problems, require selecting minimal subsets that cover a defined universe [?]. These foundational problems underlie warehouse storage assignments and similar logistical challenges due to their structural simplicity and complexity [?].

For correlated storage assignment problems in warehouses, Set Cover naturally arises when identifying minimal item clusters covering entire orders, thus improving picking efficiency [?]. CG effectively solves such problems by iteratively refining solutions through a master and a pricing subproblem [?]. Our CG implementation iteratively generates feasible product-station assignment patterns (columns) based on dual variable values, significantly enhancing computational efficiency compared to exhaustive enumeration.

4.3.1. Framework Overview

To keep the master problem tractable, we adopt a strategy where product assignment patterns are generated independently for each station. This

localized generation of feasible assignments reduces the dimensionality of the master problem, as it no longer needs to enforce product assignment uniqueness across all stations during each iteration. Instead, a diverse set of candidate patterns is generated dynamically, guided by the dual variables obtained from the master problem.

4.3.2. Master Problem Formulation

The master problem aims to select the optimal combination of station assignment patterns that collectively minimize the total number of visited stations required to fulfill all orders. The formulation ensures that:

- Each product is assigned to exactly one station (uniqueness of assignment).
- The total workload at each station does not exceed its capacity.
- Only one assignment pattern is selected for each station.

Parameters:

- δ_{psk} : Binary parameter equal to 1 if product p is included in pattern k for station s , 0 otherwise.
- z_{osk} : Binary parameter equal to 1 if any product from order o is included in pattern k for station s , 0 otherwise.

Standing assumptions: (i) Every feasible pricing pattern respects the station capacity, (i.e., $\sum_{p \in P} \delta_{psk} \leq \zeta_s \quad \forall s \in S, \forall k \in K_s$), and (ii) the total capacity equals the number of products: $\sum_{s \in S} \zeta_s = |P|$.

Decision variables:

- $y_{sk} \in \{0, 1\}$: 1 if pattern k is selected for station s , 0 otherwise.

Objective function: Minimize the total number of visited stations required across all orders:

$$\min \sum_{o \in O} \sum_{s \in S} \sum_{k \in K_s} z_{osk} \cdot y_{sk}$$

Constraints:

1. **Pattern selection constraint:** Ensure that at most one pattern is selected for each station.

$$\sum_{k \in K_s} y_{sk} \leq 1 \quad \forall s \in S \quad (\pi_s)$$

2. **Product assignment constraint:** Ensure that the universe of products is included, and since the above constraint will assure the selection of only one pattern per station (along with assumptions we gave), this constraint will then make sure that the patterns selected for each station in the end will be unique and so each product will be assigned in exactly one station.

$$\sum_{s \in S} \sum_{k \in K_s} \delta_{psk} \cdot y_{sk} \geq 1 \quad \forall p \in P \quad (\sigma_p)$$

3. **Variable domains:**

$$y_{sk} \in \{0, 1\} \quad \forall s \in S, \forall k \in K_s$$

The master problem is solved iteratively with an initial feasible set of patterns, and it will supply the pricing problem with these dual coefficients: (π_s) for pattern selection and (σ_p) for product coverage. As the solution progresses, new patterns with the potential to improve the objective value are introduced through the pricing problem.

4.3.3. Pricing Problem

The pricing problem is tasked with generating new station assignment patterns by solving a subproblem for each station. These subproblems leverage the dual values obtained from the master problem to identify assignments that can reduce the overall cost (i.e., station visits). If any new pattern has a negative reduced cost, it is added to the master problem for further optimization. This iterative process continues until no new pattern with negative reduced costs can be generated, indicating optimality with

respect to the current formulation.

Decision variable:

- $x_p \in \{0, 1\}$: 1 if product p is included in the new pattern at station s , 0 otherwise.
- $z_o \in \{0, 1\}$: 1 if order o requires visiting station s due to the new pattern, 0 otherwise. it is dependent on x_p

Objective function: Minimize the reduced cost:

$$\min \left\{ \sum_{o \in O} z_o - \sum_{p \in P} \sigma_p x_p \right\} + \pi_s$$

Constraints:

1. **Order inclusion constraints:** Ensure that z_o is set to 1 if any product from order o is included in the pattern.

$$z_o \geq x_p \quad \forall o \in O, \forall p \in P_o$$

2. **Capacity constraint:** The total number of products assigned to station s does not exceed its capacity.

$$\sum_{p \in P} x_p \leq \zeta_s$$

3. **Workload constraint:** The total workload of the pattern must not exceed the station's capacity.

$$\sum_{p \in P} x_p \cdot \frac{L_p}{V_s} \leq T_s$$

4. **Variable domains:**

$$x_p \in \{0, 1\} \quad \forall p \in P$$

$$z_o \in [0, 1] \quad \forall o \in O$$

4.3.4. *Final Optimization Step*

Upon convergence, CG yields fractional solutions requiring a final discrete assignment step. We resolve the master problem once more, enforcing binary decisions to ensure a precise assignment that meets practical warehouse constraints, product assignment uniqueness, balanced workloads, and minimal station visits.

Although accurate, this exact approach can be computationally intensive for large datasets. Future research could explore alternative post-processing techniques, such as rounding heuristics (quick approximations prioritizing feasibility), hybrid heuristic-exact methods (heuristics refined by limited exact optimization), and metaheuristic methods (genetic algorithms, tabu search, simulated annealing), which strategically search solution spaces to balance computational efficiency and solution quality for practical warehouse settings.

4.4. ***Remark: Real-World Implementation with Limited Product Reassignment***

While the Linear Programming (LP) model described in ?? provides a pathway to minimize station visits and balance workloads, fully redesigning storage assignments can be impractical in an operational warehouse. In many real-world scenarios, reassigning a large portion of products to new stations is both costly and disruptive. Moving every product to its mathematically “optimal” position may require extended downtime and significant labor, risking delays in fulfilling customer orders.

To address this concern, we introduce a practical extension that restricts the total number of products allowed to be relocated. Specifically, we limit the number of reassigned products to at most k , where k is chosen according to workforce availability, downtime constraints, or a targeted percentage of total products. By doing so, we retain the advantages of the LP formulation while mitigating its impact on everyday operations.

4.4.1. Mathematical Formulation with Limited Reassignment

Let $s'(p)$ denote the *original* station assignment of product p . In addition to the DV and constraints from ??, we add the following constraint:

$$\sum_{p \in P} x_{ps'(p)} \geq |P| - k, \quad \forall s' \in S, \quad (8)$$

where:

- $x_{p,s'(p)}$ is the original binary variable from our LP model, here used to indicate whether product p remains in its original station $s'(p)$.

Constraint (??) enforces that at least $|P| - k$ products must remain in their original stations, thereby limiting the number of relocated products to at most k . This ensures that, although the overall objective (cf. Objective (??)) aims to minimize station visits, it does so within a practical cap on how many products can be moved.

5. Experiments

This section presents the comprehensive experimental validation of our proposed methodologies. To ensure rigorous benchmarking and evaluate generalizability, we test the Heuristic, Mixed-Integer Linear Programming (MILP), and Column Generation (CG) approaches alongside newly developed meta-heuristic baselines (Genetic Algorithm and Simulated Annealing).

First, we detail the evaluation parameters and then examine the results on subsets of varying sizes, the robustness of the MILP decisions through temporal hold-outs, and sensitivity analyses.

5.1. Evaluation Methodology and Parameter Configuration

The effectiveness of our heuristic approach was rigorously evaluated against the existing product placement strategy of Company A, and later compared with the exact approach MILP model. Parameters configured for the greedy algorithm were tuned based on the Company A personalized layout and workflow strategy: *Maximum Number of Products per Community*

(*MNOPPC*) = 7, *Minimum Frequency for Pair Inclusion* (*MIN_FREQ*) = 225, *Preprocessing Frequency Threshold* (*MIN_FREQ_PREPROC*) = 90, *Ratio Threshold for Pair Relevance* (*RATIO_TO_KEEP*) = 0.2

5.2. Meta-Heuristic Benchmarking and Synthetic Scaling Validation

(To be fleshed out with the new script’s results—demonstrating performance on synthetic datasets with 500 to 5000 SKUs against GA/SA frameworks, ensuring the exact methods and CG scale properly.)

5.3. Results on the Heuristic Formulation

5.3.1. Quantitative Analysis

	Original	Heuristic
No. of visited stations	1 062 522	1 013 344
Mean of visited stations	3.730	3.557
Computation time (s)	–	45

Table 2: Heuristic solution versus original placement

The heuristic approach achieved a substantial reduction of visited stations, approximately 4.6%, with a computational time of only 45 seconds.

5.3.2. Visual Representation and Analysis

We provide graphical comparisons of Company A’s original assignment and the layout produced by our approach, where the label **Original** refers to the original placement by Company A, while **Our Solution** denotes the product placement based on our algorithm’s assignment.

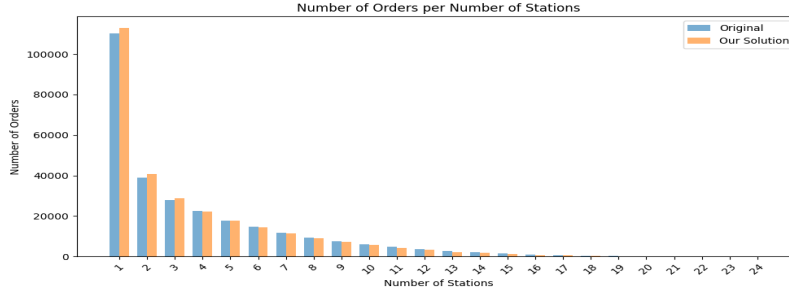


Figure 2: Number of Orders per Number of visited stations

?? shows the distribution of orders by the number of visited stations. The solution provided by our heuristic approach shifts the distribution toward fewer stops: A larger share of orders visit only a small number of stations, while the incidence of orders requiring many stations declines, consistent with the objective of minimizing station visits per order.

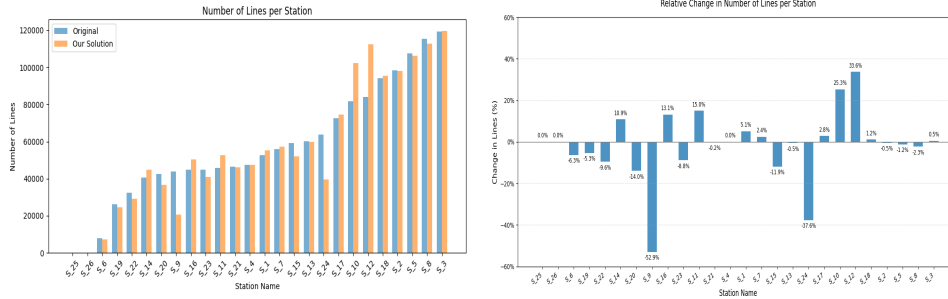


Figure 3: Number of Lines per Station (left) and Relative Percentage Change in Lines per Station (right) from heuristic assignment, compared with the Original Assignment.

Figure 3 illustrates the workload distribution across stations after applying our heuristic solution. The comparison highlights that our heuristic solution effectively reduces overall station visits, although it deviates from the "balanced workload" distribution across stations that we mentioned in this ??. Subject to management preferences regarding workload balancing and resource allocation flexibility, this solution can be still considered feasible in practice.

5.4. Results on the Mixed Integer Linear Programming Approach

This section reports results from applying our linear programming model through a commercial solver like Hexaly (i.e. LocalSolver) to address the Correlated Storage Location Assignment Problem (CSLAP). Given the dataset’s scale, we have used strategic filtering such that the model remains tractable for a large real-world instance.

5.4.1. Sub-approaches

The complete industrial instance supplied by Company A contains 21 877 Stock Keeping Units (SKUs) distributed over 26 pick-stations. Solving the MILP on this full data set is infeasible within an acceptable time on any commercial solver. We therefore applied a frequency-based pruning scheme: In our case study, SKUs with a frequency below a certain threshold (e.g., frequency = 4) are considered stable, meaning their storage positions rarely significantly affect the overall station-visit optimization. Thus, these stable SKUs are ‘frozen’ at their original assignments to reduce model complexity and improve computational efficiency without substantially compromising solution quality. Although this reduction sacrifices global optimality, it yields solutions of practical quality for real-world operations. Accordingly, we tested four sub-approaches, each distinguished by specific filtering criteria and solver configurations:

- **Sub-approach 1 (Minimal Filtering):** All single-item orders are removed and any SKU ordered fewer than four times is frozen. The resulting model contains 16 480 SKUs — **about 75% of the original catalogue**. No MIP start solution is supplied.
- **Sub-approach 2 (Minimal Filtering + MIP Start):** Uses the same instance as Sub-approach 1 (16 480 SKUs, 24 stations) but we seed the original placement of Company A as an MIP start solution to steer the solver’s initial search.
- **Sub-approach 3 (Elevated Filtering + MIP Start):** Retains only orders with more than five distinct items and freezes every SKU picked fewer than five times. The resulting MILP includes 14 416 SKUs **around 65% of the full set**. The model is again initialised with the

legacy assignment.

- **Sub-approach 4 (Elevated Filtering + MIP Start, Set DV):**
Same data set as Sub-approach 3 but reformulated in Hexaly with *set DV* rather than binary/continuous variables [?]. In Hexaly, a set decision variable is a single high-level variable that represents an unordered subset of a finite domain. We create one such variable Φ_s for every pick-station $s \in S$; its value is the subset of SKU indices assigned to that station. The usual binary matrix x_{ps} is therefore replaced by the family $\{\Phi_s\}_{s \in S}$, which reduces the DV to only 24 sets, and cleaner algebra with high level operations (e.g., *partition, contains, count*); translate directly to increased solver efficiency.

All frozen SKUs (25–35% of the total, depending on the sub-approach) remain in their original locations and continue to count towards station visits when the objective is evaluated, ensuring that the reported improvements are fully consistent with the physical warehouse.

The MILP remains extremely challenging even after pruning. To keep computation within a practical window we imposed a uniform time limit of 72,000 seconds (20 h) on every run. The shown results correspond to the best solution obtained by the solver at the end of time limit.

5.4.2. Quantitative Analysis

	Orig.	Heur.	Sub-1	Sub-2	Sub-3	Sub-4
Total visits	1 062 522	1 013 344	1 003 930	980 131	977 336	967 133
Mean visits	3.730	3.557	3.524	3.441	3.431	3.395
Time (s)	–	45	72 000	72 000	72 000	72 000

Table 4: Comparison of MILP Results under Different Sub-approaches with the Original Placement and the Heuristic Algorithm-Derived Assignment

Results underscore the value of providing a feasible initial solution (MIP start) and pruning low-frequency or low-impact items. Sub-approach 3 yields the strongest improvement in binary formulation, but the biggest

reduction is achieved in sub-approach 4. Taken together, these two settings show that targeted data pruning can materially steer the solver toward better solutions within practical time limits.

5.4.3. Visual Representation and Analysis for Final Results

The following figures compare Company A’s baseline assignment with the outcomes of the final workload-balanced model (Sub-approach 4), which delivered the best reduction.

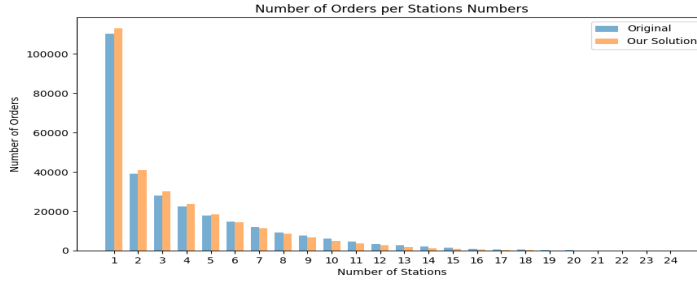


Figure 4: Number of orders per number of visited stations by sub-approach 4

Figure 4 shows the distribution of orders by the number of visited stations. The solver’s solution shifts the mass of orders to visit only a few stations and reducing orders requiring to visit to many stations, which aligns with our objective of minimizing station visits per order.

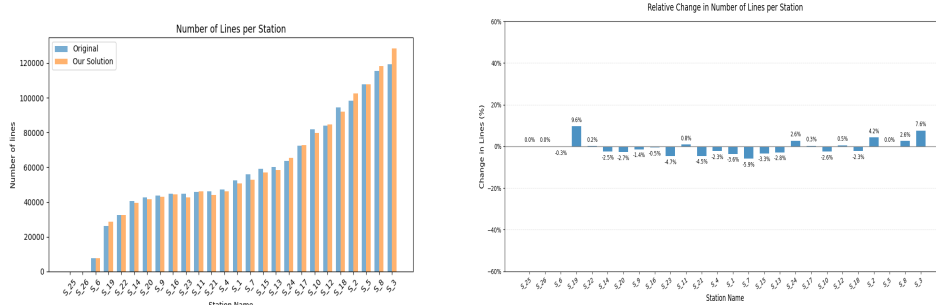


Figure 5: Number of Lines per Station (left) and Relative Percentage Change in Lines per Station (right) from solver assignment (sub-approach 4), compared with the Original Assignment.

Figure 5 illustrates the workload distribution based on the assignment from the solver on sub-approach 4. It can be observed that the distribution obtained is very close to the one derived from Company A’s original placement which is the "balanced workload" we explained in this ??, and the relative percentage-change on each station is substantially smaller compared to what we achieved through the heuristic solution, while at the same time, we have managed to minimize our objective.

Real-World Time Savings Labor Impacts: The best MILP layout (Sub-approach 4) cuts about **1 444 station visits per day**, a **9 %** drop from the original 16 098 stops/day. Over 66 working days ($\approx$ 3 months) this removes **95 304 stops**, freeing roughly **six full operating days** of picker/robot time each quarter. In practice, the site can process the same order volume almost a week faster every three months simply through better slotting.

5.5. Robustness validation via temporal hold-out

To assess the robustness of decisions made from limited historical data, we evaluate layouts *out of sample* by making optimized decision on early weeks and testing on later, unseen weeks. This temporal hold-out mirrors realistic deployment (optimize once, then operate as demand evolves) and is consistent with robustness practices in storage assignment and warehouse optimization where perfect information benchmarks (“oracle” layouts) are contrasted with implementable policies [? ?]. We performed two hold-out validation experiments on our 13-week dataset (3-month horizon):

- **Experiment 1:** Our model makes decisions knowing only the first 10 weeks of data, then it is evaluated on the remaining 3 weeks.
- **Experiment 2:** Our model makes decisions knowing only the first 8 weeks of data, then it is evaluated solely on the final 5 weeks.

These solutions are benchmarked against the original placement and against the “oracle” layout obtained when the MILP is optimized on the hold-out weeks.

Validation	Layout (Training Period)	Total Visits	Avg. Visits
Last 3 weeks	Baseline (Original placement)	265,876	4.052
	MILP (optimised on first 10 weeks)	248,358	3.785
	MILP Oracle (optimised on last 3 weeks)	233,728	3.562
Last 5 weeks	Baseline (Original placement)	423,593	3.926
	MILP (optimised on first 8 weeks)	392,318	3.636
	MILP Oracle (optimised on last 5 weeks)	380,356	3.525

Table 5: Station-visit performance evaluated on distinct hold-out validation periods.

Discussion The results of the validation experiments demonstrate the practical robustness of decisions derived from limited historical data:

Experiment 1: Using only the first 10 weeks cuts station visits by 6.6% versus the baseline, while the oracle, optimized directly on the test weeks, achieves 12.1%. This gap ($\approx 5\%$) shows that some short-term effects are missed, yet most of the benefit is already captured.

Experiment 2: With eight weeks of history, the MILP reduces visits by 7.4%; the oracle gains 10.2%. Again, historical correlations yield a strong but not exhaustive improvement. Thus, for this *specific warehouse*, the recent past provides guidance that is *good enough for meaningful savings*, although not completely predictive. However, we acknowledge such stability might vary in different operational contexts. Hence, future work should explore incorporating predictive models or stochastic scenario planning to further manage demand uncertainties.

5.6. Results on the Column Generation Approach

In this subsection, we present numerical results obtained from the implementation of our proposed CG approach and provide a comparative evaluation against the previously discussed Heuristic and MILP approaches. Due to the complex structure of our CG formulation, consisting of a single master problem and multiple pricing problems (one for each station), achieving global optimal solutions becomes computationally prohibitive when dealing

with the large-scale real-world dataset in its entirety. To effectively illustrate the merits of the CG method, we conduct our experiments on two representative subsets of the original dataset. This approach highlights the significant advantages of CG when applied to smaller datasets than typical real-world scenarios, but aligning with the scale commonly adopted in existing literature to validate theoretical aspects of optimization methodologies.

Specifically, we consider two distinct scenarios differing in scale and complexity:

- **Scenario 1:** 25 Products, 601 Orders, and 11 Stations.
- **Scenario 2:** 46 Products, 3497 Orders, and 13 Stations.

These scenarios are selected to provide insights into how the CG approach (solved using CPLEX) compares in terms of solution optimality and computational performance relative to the Heuristic and MILP (solved using Hexaly) under the sub-approach 4 conditions. The objective is to highlight the potential benefits of our CG approach in smaller-scale contexts, thus validating its applicability and efficiency as part of a multi-method optimization framework for the Correlated Storage Location Assignment Problem (CSLAP).

5.6.1. Quantitative Analysis First Scenario

	Orig.	Heur.	MILP	CG
Total number of visits	1 209	1 094	1 097	1 097
Mean number of visits	2.012	1.820	1.825	1.825
Computation Time (s)	–	2	377	80

Table 7: Comparison of CG Results under first scenario dataset with the MILP under sub-approach 4 conditions, Heuristic and Original Placement

The results presented on Table 7 show that the heuristic solution achieves the lowest total number of station visits and thus the greatest apparent reduction in our primary objective, but this approach lacks practical applicability because it is limited due to not considering a constraint to maintain

a balanced workload across stations. Conversely, both the MILP and the CG attain the global optimal solution regarding station visits, while maintaining a good workload distribution. However, a key advantage of the CG approach is evident in its computational performance; it achieves the same optimal solution as MILP but with much faster computation time.

5.6.2. Visual Representation and Analysis for First Scenario

Number of Lines Per Station: This analysis compares the workload distribution, measured by the number of retrieval lines, across each station for all evaluated approaches (Original, MILP, CG, and Heuristic) under Scenario 1. The dashed lines labeled *Optimal* represent the reference workload distribution initially established by Company A as ideal, as previously explained in this section on the Case Study ??.

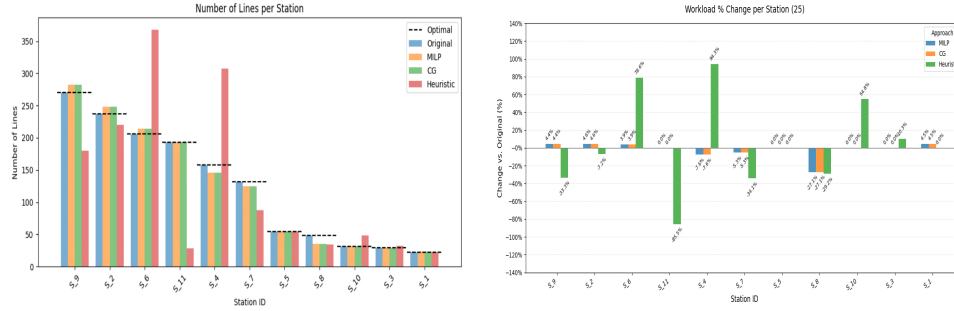


Figure 6: Number of Lines at each Station (left) and Relative Percentage Change in Lines per Station (right) based on Scenario 1.

The MILP and CG solutions yield distributions that closely track Company A’s baseline (dashed lines) while still reducing the objective. On the other hand, we check that our heuristic can achieve a great reduction in our objective, but it does not preserve a well-balanced workload in the warehouse ??.

5.6.3. Quantitative Analysis Second Scenario

	Orig.	Heur.	MILP	CG
Total number of visits	8 589	7 444	7 458	7 410
Mean number of visits	2.456	2.129	2.133	2.119
Computation Time (s)	–	3	50 000	3 258

Table 9: Comparison of CG Results under second scenario dataset with the MILP under sub-approach 4 conditions, Heuristic and Original Placement

The results presented on Table 9 highlight notable differences among the evaluated methods. The heuristic solution achieves a significant reduction in our primary objective, but its practical applicability is again limited due to its inability to maintain a balanced workload across stations. The MILP approach attains a good quality reduction in our objective while maintaining a good workload distribution, but we see that the computation time is very high. On the CG approach, the results show that we reach the highest minimum for our objective, and the key advantage is its computational performance, where we achieve better results than the MILP approach in a fraction of the computation time.

5.6.4. Visual Representation and Analysis for Second Scenario

Number of Lines Per Station: As before now we present the workload distribution across stations for Scenario 2, measured by the total number of retrieval lines.

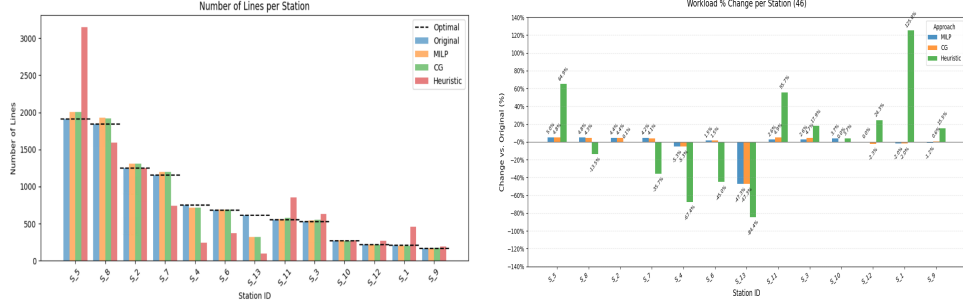


Figure 7: Number of Lines at each Station (left) and Relative Percentage Change in Lines per Station (right) based on Scenario 2.

For Scenario 2, both the MILP and CG approaches again closely approximate the optimal workload distribution, demonstrating consistency in their ability to balance station workloads effectively across larger and more complex datasets.

These experiments showed that CG matches the MILP optimum in a fraction of the time, making it the best trade-off between quality and speed in our study. The MILP approach similarly achieves high-quality solutions, but with less computational efficiency than CG, while the heuristic is fastest yet sacrifices workload balance.

Because our validation is still on limited-scale subsets, future work will investigate advanced CG enhancements (e.g., stabilization, parallelization, and alternative post-processing techniques ??) to extend the same efficiency gains to the full-size CSLAP instances of our case study.

5.7. Results on the Restricted Mixed Integer Linear Programming Approach

We now present the results of incorporating the limited reassignment constraint into our LP model. The same conditions used in Sub-approach 4 of ?? apply here, and the problem is again solved through the Hexaly commercial solver. In this scenario, we set $k = 500$, which corresponds to approximately 2.2% of the total products.

5.7.1. Quantitative Analysis

As before, the problem remains computationally intensive; hence, we specify a computation time limit (here, 72,000 seconds) and retain the best

solution found by the solver up to that threshold.

	Original	Restricted MILP
No. of station visits	1 062 522	1 060 606
Mean visits per order	3.730	3.723
computation time (s)	–	72 000

Table 11: Restricted MILP (≤ 500 re-assignments) versus original

As illustrated above the Restricted MILP approach still achieves a reduction of approximately 2,000 total station visits compared with the original layout, even though only about 2.2% of products are permitted to move. Thus, capping re-slotting effort lets managers capture most of the station-visit benefit without disruptive full-scale reassignments. By selecting an appropriate limit k to match labor, budget, and downtime constraints, warehouses can implement step-wise, “continuous-improvement” upgrades that raise throughput while leaving the vast majority of products undisturbed.

6. Conclusion

In this study, we tackled the Correlated Storage Location Assignment Problem (CSLAP) with a unique focus on minimizing station visits in automated warehouses. We evaluated three optimization methods:

- **Heuristic Approach:** A greedy clustering algorithm rapidly groups frequently co-ordered products, significantly reducing station visits. Although fast and scalable, it lacks to preserve a balanced workload, potentially creating uneven workloads.
- **Mixed Integer Linear Programming (MILP):** Our MILP formulation precisely controls workload distribution while minimizing station visits. However, computational complexity limits its scalability, requiring strategic data filtering or heuristic starts for large, real-world problems.

- **Column Generation (CG):** The CG framework decomposes the problem, iteratively refining the solution with promising assignment patterns. While maintaining balanced workloads and achieving solutions faster than MILP, it still faces complexity at very large scales.

Recognizing real-world constraints, we also introduced a practical MILP extension limiting product reassignment. This approach minimizes operational disruptions by incrementally improving warehouse layouts. Our results demonstrate substantial operational improvements (a 9% reduction or 1 444 fewer station stops daily), highlighting the practical benefit of minimizing station visits.

Managerial implications: What should operations leaders do?

- **Adopt station-visit as a KPI for slotting.** In conveyor/zone systems, station stops directly create queuing and handling delays. Track “stops per order” alongside travel time to reveal congestion hot spots that distance-only metrics miss.
- **Implement low-disruption re-slotting.** When seasonal surges or congestion dominate, use the *limited-reassignment* MILP to cap moves (e.g., limited scattering of a few top SKUs [?], $k=500$ SKUs $\approx 2.2\%$). This captures a sizable share of benefits (about 2 000 fewer stops in our case) while minimizing downtime and labor.
- **Prioritize high-impact SKU communities.** Start with communities identified by the greedy heuristic, then refine with MILP/CG under workload-balance targets to avoid creating over/under-utilized stations.
- **Recalibrate quarterly with rolling history.** The temporal hold-out analysis (Section ??) shows that layouts optimized on the recent 8-10 weeks generalize well to the next 3-5 weeks. Re-optimizing on a rolling window maintains performance with limited engineering effort.

7. Future Research Directions

Future work can enhance the realism of CSLAP solutions by incorporating heterogeneous product dimensions and storage constraints. Exploring

hybrid clustering methods combining product attributes with frequency correlations can further optimize storage assignments. Additionally, discrete-event simulations comparing original and optimized warehouse configurations will quantitatively validate our methods, highlighting operational bottlenecks, queue dynamics, and throughput improvements. Further research should integrate stochastic demand patterns and robust optimization to address uncertainty in order volumes. Real-time adaptation using machine learning could dynamically refine product placements, ensuring resilient, high-performance warehouse operations in evolving supply chain contexts.

References

- Ansari, M., Smith, J.S., 2020. Gravity clustering: A correlated storage location assignment problem approach. In: Proceedings of the 2020 Winter Simulation Conference, 1288-1299.
- Blaise, L., n.d. Modeling scheduling problems with Hexaly. Hexaly, 251 Boulevard Pereire, 75017 Paris, France.
- Caprara, A., Fischetti, M., Toth, P., 2000. Algorithms for the set covering problem. *Annals of Operations Research* 98(1-4), 353-371.
- Chabot, T., et al., 2024. Reassignment problem. In: Warehousing and Material Handling Systems for the Digital Industry, p. 137.
- Desrochers, M., Soumis, F., 1989. A column generation approach to the urban transit crew scheduling problem. *Transportation Science* 23(1), 1-13.
- Egas, C., Masel, D., 2010. Determining warehouse storage location assignments using clustering analysis. In: Proceedings of the 11th IMHRC, Milwaukee, WI.
- Garey, M.R., Johnson, D.S., 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.

- Islam, M.S., Uddin, M.K., 2023. Correlated storage assignment approach in warehouses: A systematic literature review. *Journal of Industrial Engineering and Management* 16(2), 294–318.
- Lee, I.G., Chung, S.H., Yoon, S.W. (2020). Two-stage storage assignment to minimize travel time and congestion for warehouse order picking operations. *Computers & Industrial Engineering*, 139, 106129.
- Liu, M., Poh, K.L. (2023). E-commerce warehousing: An efficient scattered storage assignment algorithm with bulky locations. *Computers & Industrial Engineering*, 181, 109236.
- Lübbecke, M.E., Desrosiers, J., 2005. Selected topics in column generation. *Operations Research* 53(6), 1007-1023.
- Pang, K.-W., Chan, H.-L., 2016. Data-mining-based algorithm for storage location assignment in a randomised warehouse. *International Journal of Production Research* 55(14), 4035-4052.
- Trindade, M.A.M., Sousa, P.S.A., Moreira, M.R.A., 2022. Ramping up a heuristic procedure for the storage location assignment problem with precedence constraints. *Flexible Services and Manufacturing Journal* 34(3), 646-669.
- Wang, M., Zhang, R.-Q., Fan, K., 2020. Improving order-picking operation through efficient storage location assignment: A new approach. *Computers & Industrial Engineering*, 139, 106186.
- West, D.B., et al., 2001. *Introduction to Graph Theory* (Vol. 2). Prentice Hall.
- Xiao, J., Zheng, L., 2010. A correlated storage location assignment problem in a single-block multi-aisle warehouse considering bill-of-material information. *International Journal of Production Research* 48(5), 1321–1338.

- Xie, L., Otten, S., 2024. Introducing combi-stations in robotic mobile fulfillment systems: A queueing-theory-based efficiency analysis. In: International Conference on Computational Logistics. Springer.
- Zhang, R.-Q., Wang, M., Pan, X. (2019). New model of the storage location assignment problem considering demand correlation pattern. *Computers & Industrial Engineering*, 129, 210-219.
- Zhang, J., Zhang, N., Tian, L., Zhou, Z., Wang, P. (2023). Robots' picking efficiency and pickers' energy expenditure: The item storage assignment policy in robotic mobile fulfillment system. *Computers & Industrial Engineering*, 176, 108918.
- Azadeh, K., De Koster, R., Roy, D., 2019. Robotized and Automated Warehouse Systems: Review and recent developments. *Transportation Science*, 53(4), 917-945.
- Boysen, N., Schwerdfeger, S., Stephan, K., 2023. A review of synchronization problems in parts-to-picker warehouses. *European Journal of Operational Research*, 307(3), 1374-1390.
- Kim, T., Hong, S., 2020. Two Picker Cooperation Strategies for Zone Picking Systems With PTL Technology. *IEEE Xplore*, <https://ieeexplore.ieee.org/document/9106329>
- Mirzaei, M., Zaerpour, N., De Koster, R., 2021. How to benefit from order data: correlated dispersed storage assignment in robotic warehouses. *International Journal of Production Research*, 299(2), 522-540.
- Muter, I., Oncan, T., 2022. Order batching and picker scheduling in warehouse order picking. *Institute of Industrial and Systems Engineers, IISE Transactions*, 299(1), 127-142.
- Pan, J.C.H., Shih, P.H., Wu, M.H., 2015. Order batching in a pick-and-pass warehousing system with group genetic algorithm. *Omega*, 57, 238-248.

- Tarczyński, G., 2023. Linear programming models for optimal workload and batching in pick-and-pass warehousing systems. *Operations Research and Decisions*, DOI:10.37190/ord230309 Authors:
- van Gils, T., Ramaekers, K., Caris, A., de Koster, R., 2018. Designing efficient order picking systems by combining planning problems: State-of-the-art classification and review. *European Journal of Operational Research*, 267(1), 1-15.
- Xie, L., Thieme, N., Krenzler, R., Li, H., 2021. Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems *European Journal of Operational Research*, <https://doi.org/10.1016/j.ejor.2020.05.032>.
- Zhen, L., Tan, Z., de Koster, R., He, X., Wang, S., Wang, H., 2025. Optimizing Warehouse Operations with Autonomous Mobile Robots. *Transportation Science*, forthcoming.
- Zhu, S., Hu, X., Huang, K., Yuan, Y., 2021. Optimization of product category allocation in multiple warehouses to minimize splitting of online supermarket customer orders. *European Journal of Operational Research*, 291(2), 642-658.